

Smart Travel Companion

AI-ассистент, который превращает покупку билета
в операционный штаб семейной поездки

Алматы → Астана

ночной поезд · семья из 4

Клиент купил билет — и ушёл

- **Halyk Travel сегодня** — сервис покупки билетов. На этом контакт с клиентом заканчивается.
- **ОТЕЛЬ, страховка, трансфер, развлечения, питание** клиент организует сам, в чужих приложениях.
- **Все downstream-транзакции** уходят к конкурентам — вместе с данными и лояльностью.

1

транзакция вместо 15+

Каждая неоформленная услуга — упущенная выручка и ушедший клиент.

Продукт строится на карте тревог клиента, а не на списке сервисов

- Каждое сообщение отвечает на конкретную мысль клиента в конкретный момент.
- «Когда предлагать Arpteka» и «когда родитель думает об аптечке» — разные вопросы.
- Клиент не чувствует, что ему продают, — он чувствует, что о нём заботятся.

Из разовой покупки — в штаб всей поездки

- **Halyk ведёт клиента** от покупки билета до возвращения домой, закрывая каждую тревогу в нужный момент.
- **~18 уведомлений за 18 дней** — примерно одно в день. Умное молчание важнее умных сообщений.



Одна семья, выходные в Астане

A Айдар организатор, плательщик

Ал Алия удостоверение истекает 28 июля

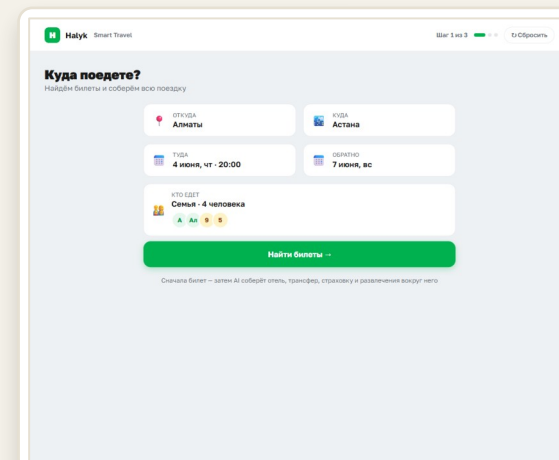
9 Айша 9 лет

5 Тимур 5 лет

Маршрут: **Алматы → Астана, ночной поезд ~13 ч**

Даты: **4→5 июня туда · 5–7 июня в Астане · дождливая суббота +14°**

- **Главный страх организатора:** «что-то не предусмотреть».
- **Готов платить** за снятие тревоги, а не за сами сервисы.

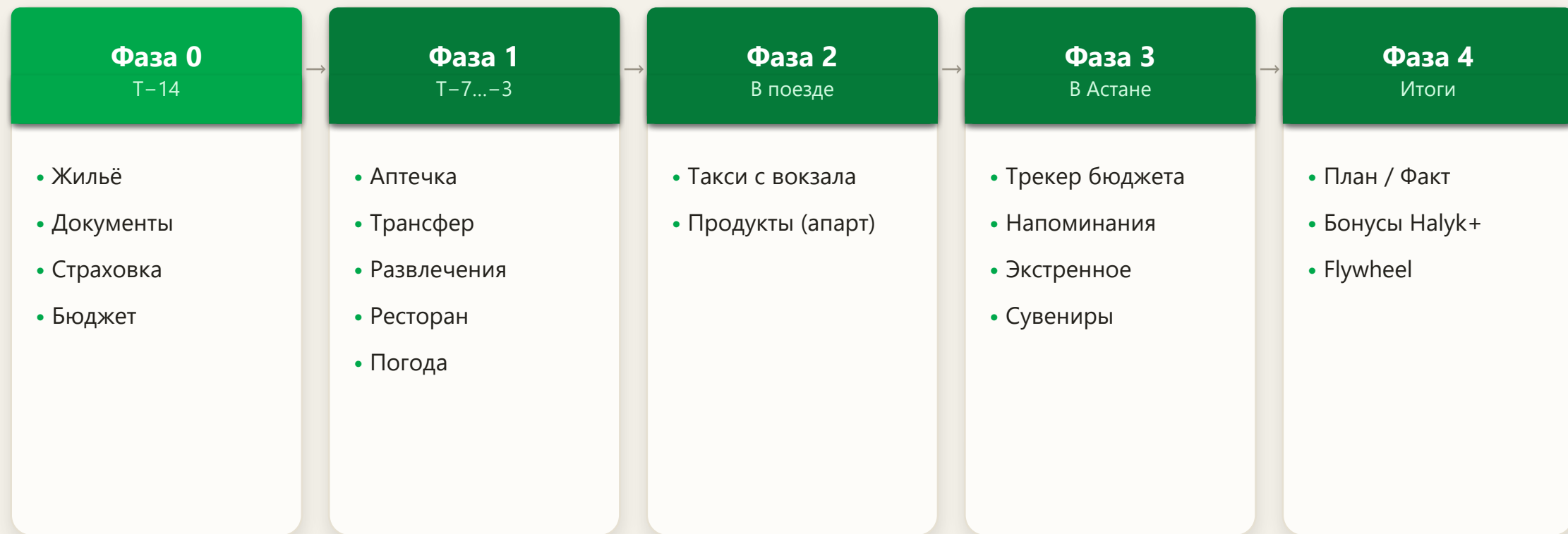


Демо открывается с покупки билета (3 шага)

Карта тревог клиента

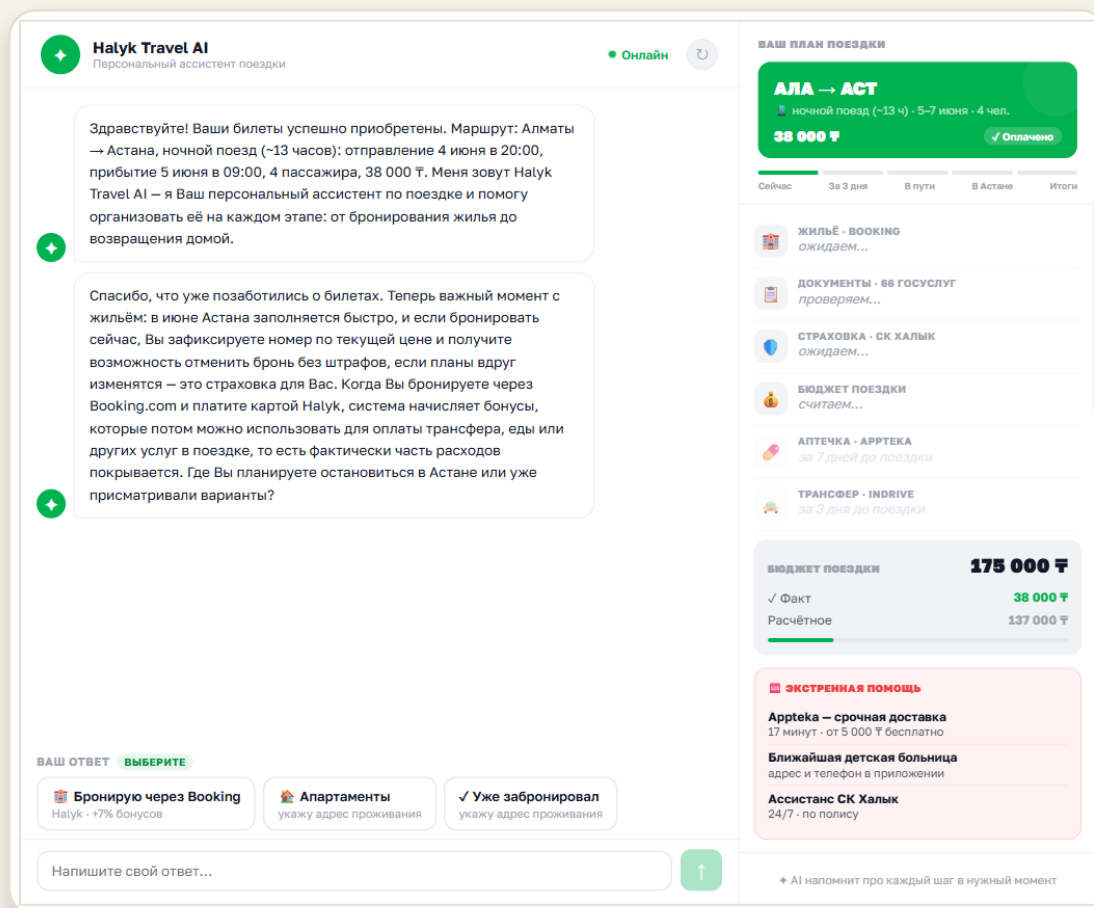
Момент	Мысль в голове	Решение Halyk
Сразу после билета	«Надо отель — разберут быстро»	Booking.com
После отеля	«Документы в порядке?»	66 госуслуг
После отеля	«Страховка нужна, наверное»	СК Халык
После отеля	«Сколько вообще уйдёт денег?»	Автосмета
За 7 дней	«Аптечку собрать надо»	Appteka
За 3 дня	«Как с вокзала доедем?»	inDrive
За 3 дня	«Чем занять детей в дождь?»	Kino.kz
В поездке	«Сколько уже потратили?»	Трекер бюджета
После поездки	«Сколько вышло в итоге?»	Итоги план/факт

Путь клиента: 10 этапов, 5 фаз



Каждый пункт: **locked** → **wait** → **done**. Разблокировка — автоматически при смене фазы.

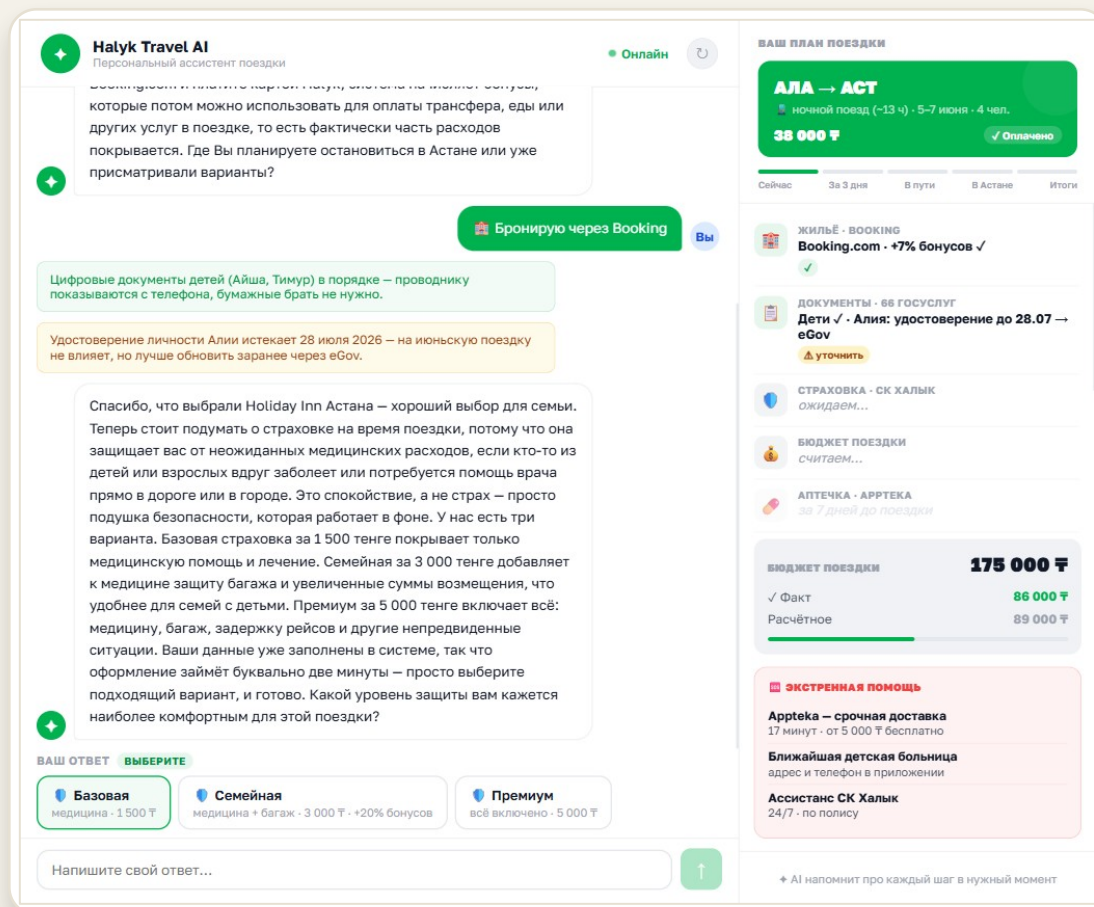
Билет — точка входа. Дальше AI собирает поездку



Старт companion после покупки билета

- **3-шаговый визард** покупки билета → передаёт клиента в companion.
- **Первое сообщение:** билеты куплены, факты поездки, «я — ваш ассистент».
- **Второе:** вопрос про жильё — Booking.com и бонусы Halyk.
- **Справа — живой Travel Plan** из 10 пунктов и бюджет.

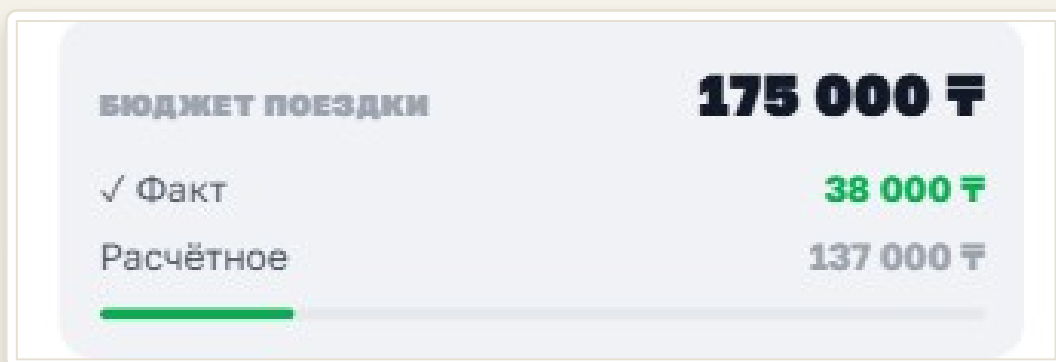
Проактивная проверка документов



Алерт о документах появляется сам

- **Halyk сам проверяет** документы семьи через 66 госуслуг — молча.
- **Замечает:** удостоверение Алии истекает 28 июля → предлагает eGov заранее.
- **Только Halyk знает эти данные.** Конкурент этого не сделает.
- **Одно такое уведомление** строит больше доверия, чем любой кешбэк.

Сквозной бюджет: «всё под контролем»

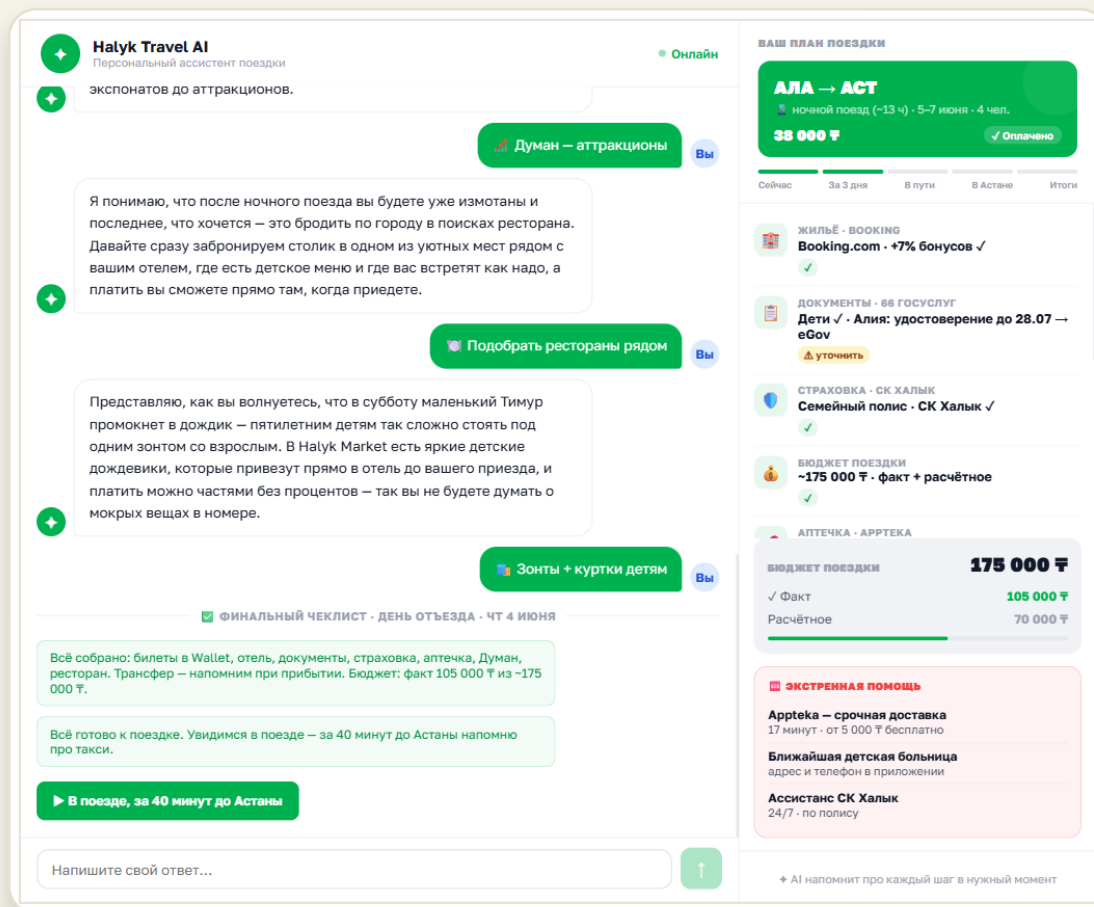


Панель бюджета на старте поездки

Факт растёт: **38 000 → 89 000 → 105 000 ₸**
Расчётное тает, итог держится **~175 000 ₸**

- **Два уровня:** Факт (оплачено/зарезервировано) + Расчётное (прогноз системы).
- **Смета собирается сама** по известным данным — клиент ничего не вводит.
- **По мере оплат** деньги перетекают из расчётного в факт, итог стабилен.
- **Это эмоциональный продукт**, не аналитический: клиент перестаёт беспокоиться о деньгах.

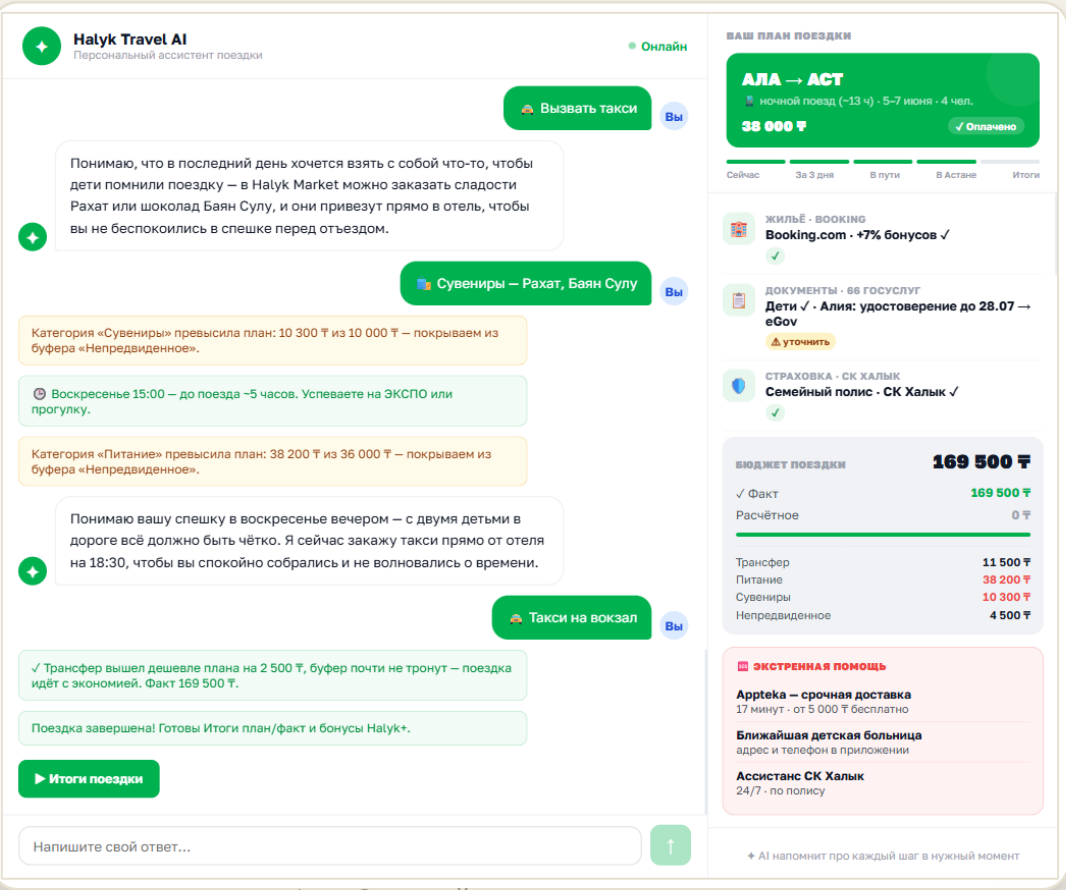
Погода меняет план поездки



Фаза 1: подготовка под дождь

- Система мониторит прогноз с момента покупки билета.
- Дождливая суббота +14° → только крытые активности (Думан), дождевики, такси.
- Рекомендации адаптируются: дождь · жара · холод · норма — разные сценарии.
- В демо — дождевой сценарий: самая богатая логика.

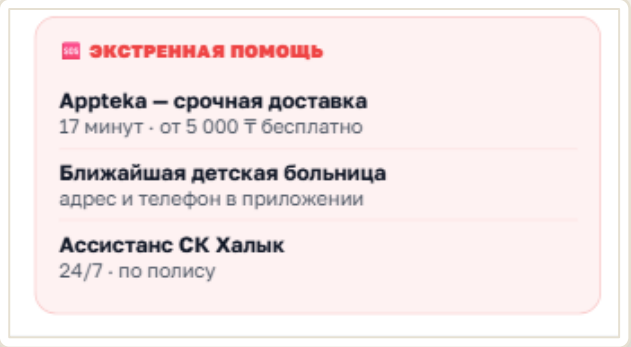
В пути и на месте: момент важнее экрана



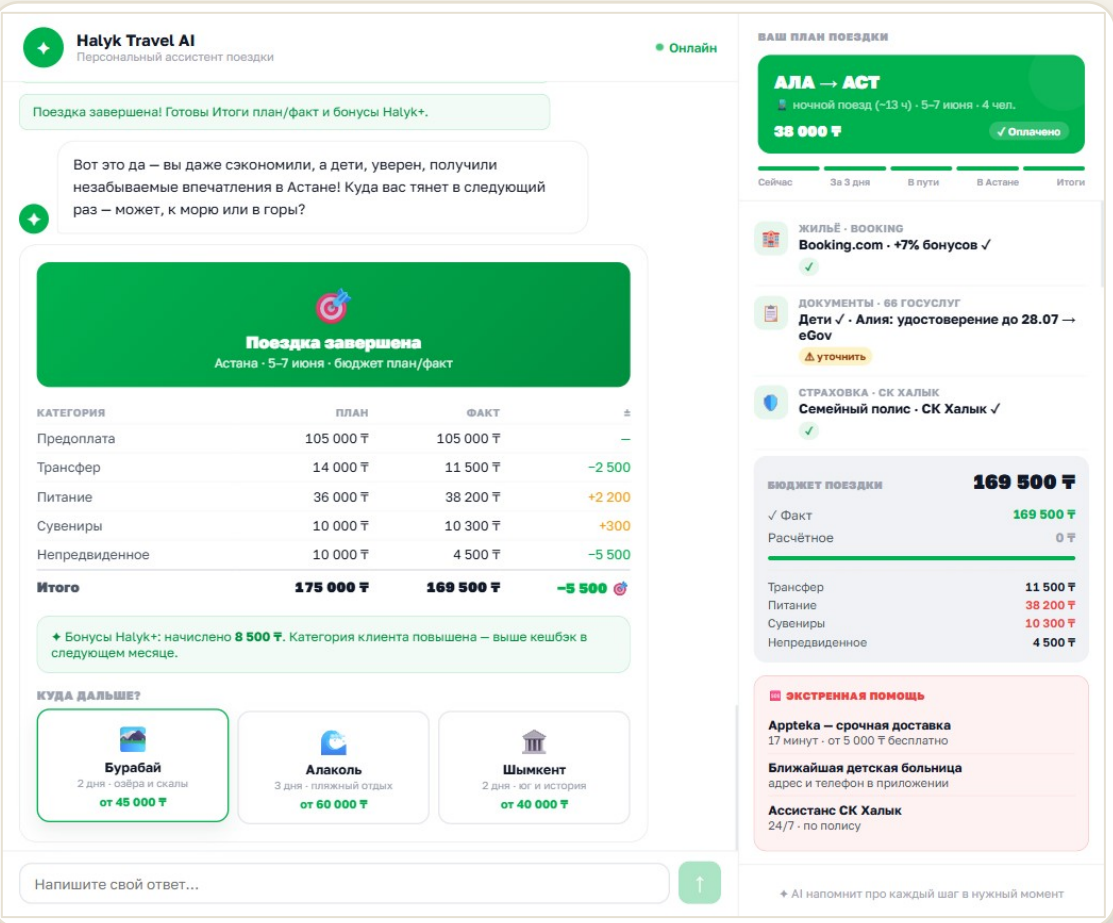
Фаза 3: живой трекер и напоминания

- **За 10 минут до Астаны** — такси одним тапом (клиент стоит с детьми и чемоданами).
- **Живой трекер:** платежи картой Halyk сами падают в категории бюджета.

Экстренный блок — всегда под рукой:



Итоги план/факт и новый виток



Экран Итоги + Flywheel

- **План 175 000 → Факт 169 500 ₸ (-5 500).** Уложились в бюджет 🎯.
- **Бонусы Halyk+ начислены,** категория клиента повышена.
- **Три варианта следующей поездки —** Бурабай, Алаколь, Шымкент.
- **Один тап → Halyk Travel с поиском.** Цикл (flywheel) замыкается.

Каждая тревога — точка монетизации

Booking.com

revenue share + бонусы Halyk

СК Халык

страховая премия

Appteka · inDrive · Airba

revenue share с партнёров

Kino.kz

прямой доход (сервис Halyk) + бонусы

Halyk Market

продажа + рассрочка Homebank

Halyk Pay / QR

interchange + данные о тратах

66 госуслуг

доверие и retention

Halyk+

лояльность → flywheel

Клиент остаётся внутри экосистемы Halyk всю поездку — не уходит к конкурентам.

Три ключевых вывода

1

Тревоги, не сервисы

Продукт отвечает на конкретную мысль клиента в нужный момент времени, а не предлагает услуги по календарю.

2

Проактивная проверка

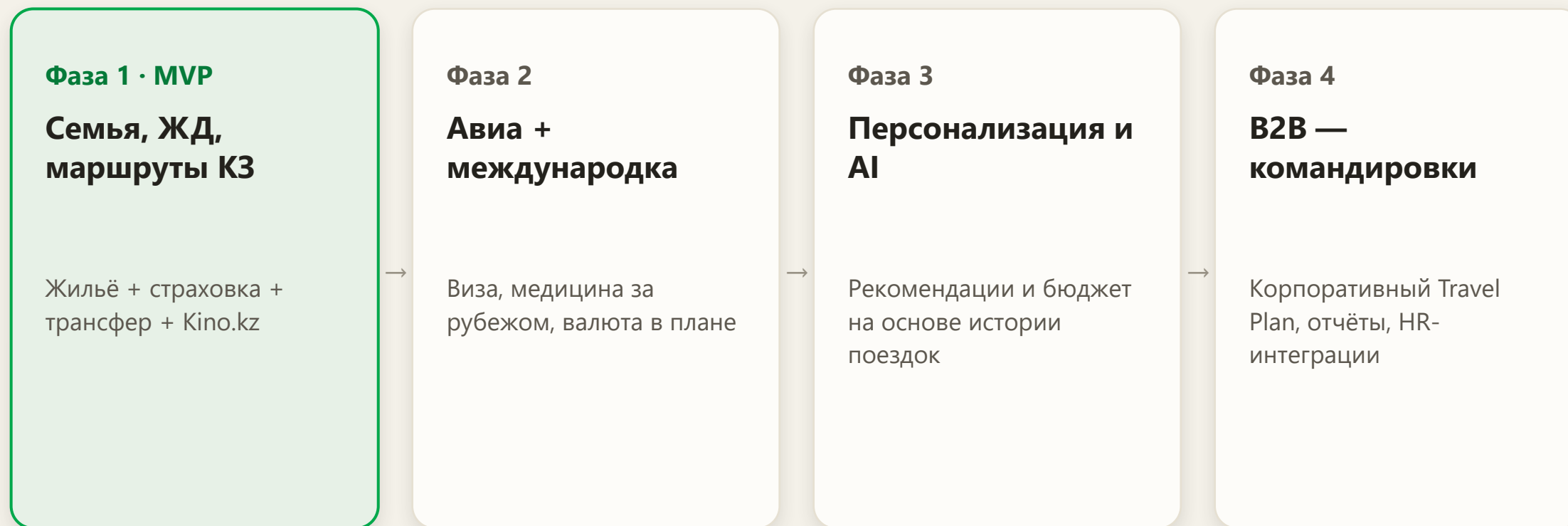
Документы и погода — преимущество, доступное только Halyk: данные семьи уже в экосистеме.

3

Трекер = стимул карты

Полная картина трат только по транзакциям Halyk. Наличные не видны — клиент сам хочет платить картой.

От семьи на поезде — к B2B-командировкам

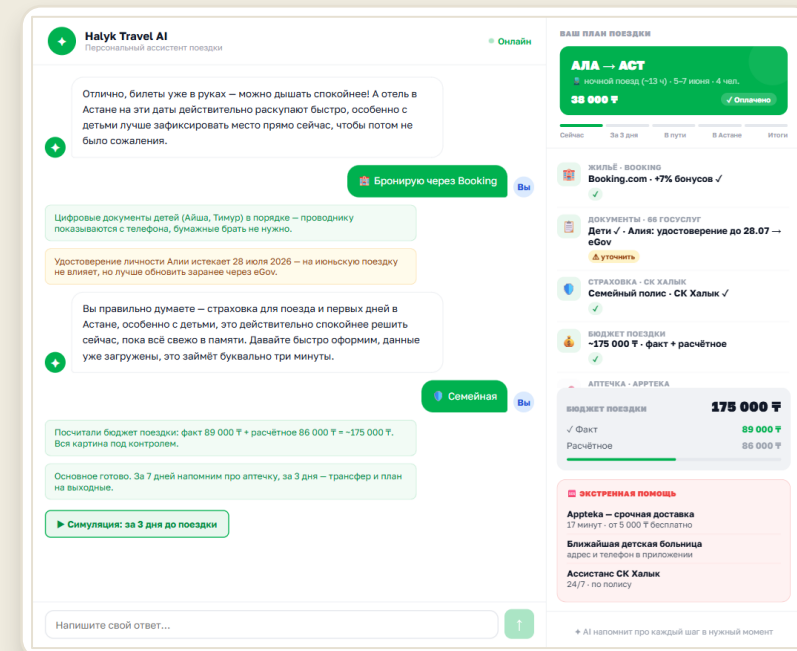


ДЕМО

Демо вживую

docker compose up → localhost:8000

- Визард покупки билета → передача в companion.
- Жильё → проактивный алерт о документах → страховка → бюджет.
- Погодная адаптация, живой трекер, экстренный блок.
- Итоги план/факт и Flywheel.



Работает полностью оффлайн — fallback на каждом шаге, сеть не нужна.

Резервный скриншот на случай сбоя

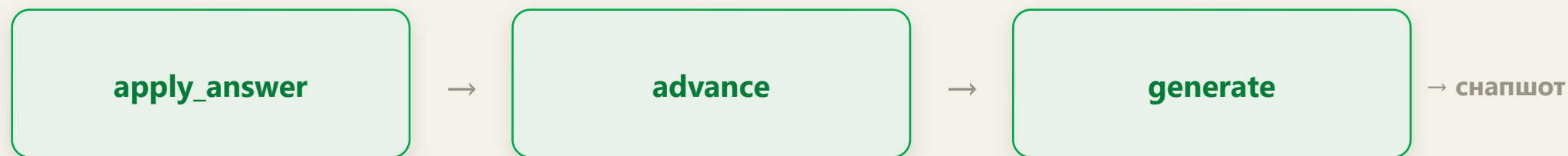
Один контейнер, весь путь end-to-end



Django ORM (SQLite) — единый источник правды. WhiteNoise отдаёт SPA. Всё в одном Docker-образе.

- **Стек:** Django 5 + DRF · React 18 / Vite / TS · SQLite · LangChain/LangGraph · Gunicorn · Docker.
- **Запуск:** docker compose up → localhost:8000, end-to-end в один шаг.
- **Демо работает полностью оффлайн:** у каждого AI-шага есть готовый fallback.

Граф — это и есть продукт



- **Декларативный сценарий:** фазы → шаги → чипсы. Контент отделён от движка.
- **Human-in-the-loop:** клиент отвечает чипсами, граф делает один ход и ждёт.
- **Stateless поверх Django ORM** — единый источник правды, без чекпойнтера LangGraph.
- **Состояние после рестарта** = phase + step_index + plan + budget + messages. Переживает перезапуск и refresh.

Язык тревоги, а не продажи

- **claude-haiku-4-5** генерирует каждое сообщение под контекст: погода, дети, отель.
- **Тон официальный, на «Вы»;** схема explain-then-offer (сначала зачем, потом предложение).
- **На любой ошибке или без ключа** → канонический fallback-текст шага.
- **Поэтому поездка всегда доходит до конца** — даже без сети и без API-ключа.

Принцип системного промпта

«Каждое сообщение отвечает на конкретную мысль-тревогу клиента — он чувствует заботу, не продажу. Стиль официальный, на «Вы». Без нагнетания.»

→ fallback гарантирует, что демо никогда не «падает» из-за сети.

Нalyk перестаёт продавать билеты — и начинает вести клиента.

От покупки билета до возвращения домой — каждая тревога закрыта в нужный момент.

Спасибо! Демо: `docker compose up` → `localhost:8000`